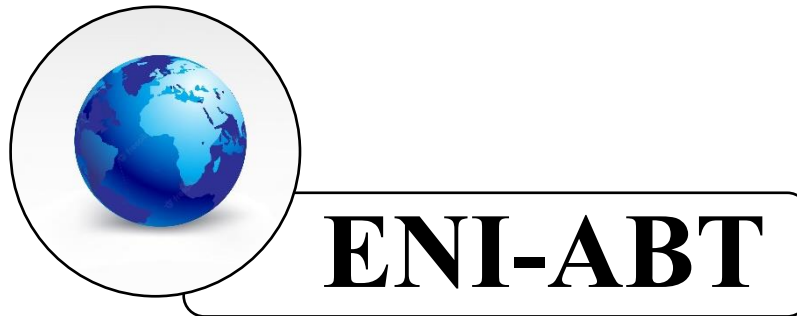




ECOLE NATIONALE D'INGENIEURS ABDERHAMANE BABA TOURE (ENI-ABT)



Rapport de Mini projet

**Thème : Conception et Développement d'un Système Automatisé
d'Information pour la Gestion Commerciale d'une Librairie**

Formation : Master en Management des Systèmes d'Information (MSI)

Département d'Enseignement et de la Recherche (DER)

Génie Informatique et des Télécommunications (GIT)

Encadrant :

Dr. Doumbia

Membres :

Drissa Sidiki TRAORE

Bagaoussou COULIBALY

Souleymane COULIBALY

Fatoumata DIAKITE

Pascal DIARRA

Table of Contents

Introduction.....	4
Structure des fichiers de la base de données.....	4
Script de création de la base de données et des tables	5
Programmes de saisie, d'édition et de mise à jour de la base de données.....	8
Table distributeurs.....	8
Saisie (INSERT)	8
Édition (SELECT).....	8
Mise à jour (UPDATE).....	8
Table livres.....	8
Saisie.....	8
Édition.....	8
Mise à jour.....	8
Table commandes.....	8
Saisie.....	8
Édition.....	8
Mise à jour.....	9
Table lignes_commande.....	9
Saisie.....	9
Édition.....	9
Mise à jour.....	9
Table livraisons	9
Saisie.....	9
Édition.....	9
Mise à jour.....	9
Table lignes_livraison.....	9
Saisie.....	9
Édition.....	10
Mise à jour.....	10
Table achats.....	10
Saisie.....	10
Édition.....	10
Mise à jour.....	10
Table ventes.....	10
Saisie.....	10
Édition.....	10

Mise à jour	10
Table lignes_vente	10
Saisie.....	10
Édition.....	11
Mise à jour	11
Conclusion	11

Introduction

Dans le cadre de ce projet, nous abordons la phase opérationnelle et physique de la conception d'une base de données, étape essentielle pour passer de la modélisation théorique à une implémentation concrète et fonctionnelle. Après avoir défini les modèles conceptuel et logique, il s'agit à présent de développer le Modèle Physique des Données (MPD) ainsi que le Modèle Opérationnel et Technique (MOpT) dans un environnement choisi.

Pour cette réalisation, le logiciel **MySQL Workbench** a été utilisé afin de générer la structure de la base, visualiser les schémas physiques, exécuter les scripts de création et assurer une transition fiable entre les modèles et leur implémentation.

Cette étape consiste à définir la structure réelle des fichiers de la base, à créer les tables et contraintes, puis à mettre en place les programmes permettant les opérations de saisie, de consultation, de modification et de mise à jour des données. L'objectif de cette phase est d'assurer la réalisation d'un système fiable, cohérent et exploitable, capable de répondre aux besoins identifiés lors des étapes précédentes de la conception.

Structure des fichiers de la base de données

La structure des fichiers de la base de données correspond à l'ensemble des objets physiques réellement créés dans le Système de Gestion de Base de Données (SGBD) retenu. Elle constitue la traduction concrète du Modèle Physique des Données (MPD) et regroupe l'ensemble des tables, clés primaires, clés étrangères, contraintes d'intégrité et types de données définis lors de la conception.

Dans notre projet, la base de données a été générée à partir d'un script SQL complet. Ce script assure la création automatique de l'ensemble de l'architecture nécessaire : création du schéma, définition des tables, déclaration des relations entre entités, ainsi que l'ajout des contraintes permettant de garantir la cohérence et l'intégrité des données.

Chaque table représente une entité du système d'information et contient les champs correspondant à ses attributs. Les types de données sont définis en fonction des besoins réels de stockage, et les clés primaires permettent d'identifier de manière unique chaque enregistrement. Les clés étrangères, quant à elles, assurent le respect des relations entre tables conformément au modèle logique établi.

La structure des fichiers ainsi définie garantit une organisation claire, cohérente et optimisée pour les opérations de saisie, de consultation et de mise à jour qui seront prises en charge dans les sections suivantes. L'ensemble du script de création de la base et des tables est

fourni en annexe et sont également présenté pour illustrer précisément la structure physique mise en place.

Script de création de la base de données et des tables

```
DROP DATABASE IF EXISTS librairie;  
CREATE DATABASE librairie CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE librairie;
```

Figure 1: Création de la base

```
-- =====  
-- 1. Table distributeurs  
-- =====  
CREATE TABLE distributeurs (  
    id_distributeur INT AUTO_INCREMENT PRIMARY KEY,  
    code_distributeur VARCHAR(20) NOT NULL UNIQUE,  
    raison_sociale VARCHAR(100) NOT NULL,  
    adresse VARCHAR(255) NULL,  
    telephone VARCHAR(20) NULL  
) ENGINE=InnoDB;  
  
-- =====  
-- 2. Table livres  
-- =====  
CREATE TABLE livres (  
    id_livre INT AUTO_INCREMENT PRIMARY KEY,  
    code_livre VARCHAR(30) NOT NULL UNIQUE, -- ISBN ou code interne  
    titre VARCHAR(255) NOT NULL,  
    auteurs VARCHAR(255) NOT NULL, -- séparés par ; ou ,  
    editeur VARCHAR(100) NOT NULL,  
    prix_vente_ht DECIMAL(10,2) NOT NULL CHECK (prix_vente_ht > 0),  
    stock_securite INT NOT NULL DEFAULT 5,  
    quantite_en_stock INT NOT NULL DEFAULT 0  
) ENGINE=InnoDB;
```

```
-- =====
-- 3. Table commandes (vers les distributeurs)
-- =====
CREATE TABLE commandes (
  id_commande      INT AUTO_INCREMENT PRIMARY KEY,
  id_distributeur  INT NOT NULL,
  date_commande    DATE NOT NULL DEFAULT (CURRENT_DATE),
  statut           ENUM('EN_COURS', 'PARTIELLEMENT_LIVREE', 'TERMINEE') NOT NULL DEFAULT 'EN_COURS',

  FOREIGN KEY (id_distributeur) REFERENCES distributeurs(id_distributeur)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- =====
-- 4. Table lignes_commande
-- =====
CREATE TABLE lignes_commande (
  id_ligne_commande INT AUTO_INCREMENT PRIMARY KEY,
  id_commande        INT NOT NULL,
  id_livre            INT NOT NULL,
  quantite_commandee INT NOT NULL CHECK (quantite_commandee > 0),
  quantite_livree     INT NOT NULL DEFAULT 0,                -- cumul des livraisons reçues

  FOREIGN KEY (id_commande) REFERENCES commandes(id_commande)
    ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (id_livre) REFERENCES livres(id_livre)
    ON DELETE RESTRICT ON UPDATE CASCADE,

  UNIQUE KEY uniq_cmd_livre (id_commande, id_livre)          -- évite doublons dans même commande
) ENGINE=InnoDB;
```

```
-- =====
-- 5. Table livraisons (réceptions)
-- =====
CREATE TABLE livraisons (
  id_livraison      INT AUTO_INCREMENT PRIMARY KEY,
  id_commande        INT NOT NULL,
  date_livraison     DATE NOT NULL,
  num_bon_livraison_dist VARCHAR(50) NOT NULL UNIQUE,

  FOREIGN KEY (id_commande) REFERENCES commandes(id_commande)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- =====
-- 6. Table lignes_livraison
-- =====
CREATE TABLE lignes_livraison (
  id_ligne_livraison INT AUTO_INCREMENT PRIMARY KEY,
  id_livraison        INT NOT NULL,
  id_livre            INT NOT NULL,
  quantite_livree     INT NOT NULL CHECK (quantite_livree > 0),
  prix_achat_unitaire_ht DECIMAL(10,2) NOT NULL CHECK (prix_achat_unitaire_ht > 0),

  FOREIGN KEY (id_livraison) REFERENCES livraisons(id_livraison)
    ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (id_livre) REFERENCES livres(id_livre)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```

-- =====
-- 7. Table achats (pour la comptabilité - 1 ligne = 1 ligne de livraison valide)
-- =====
CREATE TABLE achats (
    id_achat          INT AUTO_INCREMENT PRIMARY KEY,
    id_ligne_livraison INT NOT NULL,
    date_achat        DATE NOT NULL,                -- date de signature du bon

    FOREIGN KEY (id_ligne_livraison) REFERENCES lignes_livraison(id_ligne_livraison)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    UNIQUE KEY uniq_ligne_livraison (id_ligne_livraison) -- une seule entrée comptable
) ENGINE=InnoDB;

-- =====
-- 8. Table ventes (tickets de caisse)
-- =====
CREATE TABLE ventes (
    id_vente          INT AUTO_INCREMENT PRIMARY KEY,
    date_vente        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    total_ht          DECIMAL(10,2) NOT NULL,
    total_ttc         DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;

```

```

-- =====
-- 9. Table lignes_vente
-- =====
CREATE TABLE lignes_vente (
    id_ligne_vente     INT AUTO_INCREMENT PRIMARY KEY,
    id_vente           INT NOT NULL,
    id_livre           INT NOT NULL,
    quantite_vendue    INT NOT NULL CHECK (quantite_vendue > 0),
    prix_vente_unitaire_ht DECIMAL(10,2) NOT NULL,

    FOREIGN KEY (id_vente) REFERENCES ventes(id_vente)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (id_livre) REFERENCES livres(id_livre)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- =====
-- Index utiles (performances)
-- =====
CREATE INDEX idx_livres_titre    ON livres(titre);
CREATE INDEX idx_livres_code    ON livres(code_livre);
CREATE INDEX idx_commandes_date ON commandes(date_commande);
CREATE INDEX idx_ventes_date    ON ventes(date_vente);

```

Programmes de saisie, d'édition et de mise à jour de la base de données

Table distributeurs

Saisie (INSERT)

```
INSERT INTO distributeurs (code_distributeur, raison_sociale, adresse, telephone)
VALUES ('DIST001', 'Société LivrePlus', 'Aci 2000, Bamako', '70 00 00 00');
```

Édition (SELECT)

```
SELECT * FROM distributeurs;
```

Mise à jour (UPDATE)

```
UPDATE distributeurs
SET telephone = '75 55 55 55'
WHERE id_distributeur = 1;
```

Table livres

Saisie

```
INSERT INTO livres (code_livre, titre, auteurs, editeur, prix_vente_ht, stock_securite,
quantite_en_stock)
VALUES ('ISBN12345', 'Programmation Java', 'K. Traoré', 'Dunod', 15000, 5, 20);
```

Édition

```
SELECT * FROM livres;
```

Mise à jour

```
UPDATE livres
SET quantite_en_stock = quantite_en_stock + 10
WHERE id_livre = 1;
```

Table commandes

Saisie

```
INSERT INTO commandes (id_distributeur, date_commande, statut)
VALUES (1, CURRENT_DATE, 'EN_COURS');
```

Édition

```
SELECT * FROM commandes;
```


Mise à jour

UPDATE commandes

SET statut = 'PARTIELLEMENT_LIVREE'

WHERE id_commande = 1;

Table lignes_commande

Saisie

INSERT INTO lignes_commande (id_commande, id_livre, quantite_commandee)

VALUES (1, 1, 30);

Édition

SELECT * FROM lignes_commande;

Mise à jour

UPDATE lignes_commande

SET quantite_livree = quantite_livree + 5

WHERE id_ligne_commande = 1;

Table livraisons

Saisie

INSERT INTO livraisons (id_commande, date_livraison, num_bon_livraison_dist)

VALUES (1, CURRENT_DATE, 'BL2025-001');

Édition

SELECT * FROM livraisons;

Mise à jour

UPDATE livraisons

SET date_livraison = '2025-11-20'

WHERE id_livraison = 1;

Table lignes_livraison

Saisie

INSERT INTO lignes_livraison (id_livraison, id_livre, quantite_livree, prix_achat_unitaire_ht)

VALUES (1, 1, 10, 9000);

Édition

```
SELECT * FROM lignes_livraison;
```

Mise à jour

```
UPDATE lignes_livraison  
SET quantite_livree = 20  
WHERE id_ligne_livraison = 1;
```

Table achats

Saisie

```
INSERT INTO achats (id_ligne_livraison, date_achat)  
VALUES (1, CURRENT_DATE);
```

Édition

```
SELECT * FROM achats;
```

Mise à jour

```
UPDATE achats  
SET date_achat = '2025-11-22'  
WHERE id_achat = 1;
```

Table ventes

Saisie

```
INSERT INTO ventes (total_ht, total_ttc)  
VALUES (30000, 36000);
```

Édition

```
SELECT * FROM ventes;
```

Mise à jour

```
UPDATE ventes  
SET total_ht = 32000, total_ttc = 38400  
WHERE id_vente = 1;
```

Table lignes_vente

Saisie

```
INSERT INTO lignes_vente (id_vente, id_livre, quantite_vendue, prix_vente_unitaire_ht)
```

```
VALUES (1, 1, 2, 15000);
```

Édition

```
SELECT * FROM lignes_vente;
```

Mise à jour

```
UPDATE lignes_vente
```

```
SET quantite_vendue = 3
```

```
WHERE id_ligne_vente = 1;
```

Conclusion

Ce mini-projet a permis de parcourir l'ensemble du processus de conception d'un système d'information, depuis l'analyse conceptuelle jusqu'à la mise en œuvre physique. Dans un premier temps, l'élaboration du dictionnaire des données, de la SAT, du MCD et du MCT a permis de clarifier les besoins, de structurer l'information et de formaliser les traitements. Ces étapes conceptuelles ont jeté les bases nécessaires pour garantir la cohérence du système.

Le passage au niveau logique a ensuite consisté à traduire les modèles conceptuels en modèles plus directement exploitables par un SGBD. Cette phase a assuré une transition rigoureuse entre la représentation abstraite des données et leur organisation relationnelle, permettant ainsi de préparer efficacement la réalisation technique.

Enfin, le niveau opérationnel a permis de matérialiser le système d'information dans l'environnement choisi. La construction du script de création de la base, la définition de la structure physique des fichiers ainsi que le développement des programmes de saisie, d'édition et de mise à jour ont concrétisé l'ensemble du travail précédent.

Ce projet offre ainsi une vision complète et progressive de la démarche de conception d'une base de données, depuis la réflexion initiale jusqu'à la mise en œuvre fonctionnelle. Il constitue une expérience pratique essentielle pour maîtriser les différentes étapes du développement d'un système d'information fiable, cohérent et opérationnel.